

CYNAR: A Practical Guide

Marco Baiesi

Ivan Di Terlizzi

Abstract

This document collects some practical considerations on the `cynar` package, which were born empirically, while building and validating the four example systems shipped with `cynar.simul` (`linear`, `hair`, `Lorenz`, `Lorenz96`). They include how to choose a fitting window for the traffic estimator, how the grid-based inflow-rate and v^2 -method estimators can fail and how the package now guards against it, how the optional smoothing step is configured, and a system-by-system reference table for every example distributed with the package. We assume that the reader is familiar with the arxiv paper¹. The recommendations below come from these four systems, where the true σ , $\mathcal{T}$, and $\mathcal{G}$ are known; treat them as a starting point on new data, not as a guarantee.

Contents

1	Traffic estimation	2
1.1	The exact identity behind the correlation-derivative estimator	2
1.2	Model order (N_e , N_g): richer is not safer	2
1.3	The fit window (k_1, k_2, H) is system-dependent	2
1.4	Automating the choice: minimizing across-chunk variance is a useful default, not a guarantee	3
2	Inflow-rate estimation	3
2.1	Grid-boundary score artifacts (fixed)	3
2.2	The safety-net threshold does not scale with D	4
2.3	Choosing Δx for a swept parameter family	4
2.4	No universal blind heuristic for Δx without ground truth	4
2.5	A starting point for the sweep, on unfamiliar data	4
3	Smoothing	5
3.1	The smoothed-field construction	5
3.2	Bandwidth must be tied to cell size, not to the raw data's spread (fixed)	5
3.3	Smoothing over sparse or fractal supports needs occupied-cells-only neighbors (fixed, with a residual limitation)	6
4	The v^2-method	6
5	Partial observation	7
6	Per-system configuration reference	7

¹M. Baiesi and I. Di Terlizzi, *CYNAR: a trajectory-based estimator of entropy production*, arXiv:2609.39654.

1 Traffic estimation

1.1 The exact identity behind the correlation-derivative estimator

`cynar.traffic` estimates the diffusion tensor as $\mathbf{D} = -\dot{\mathbf{C}}_*(0^+)$, the (negative) short-time derivative of the symmetrized correlation matrix. This identity is *exact* for any stationary diffusion with state-independent $\mathbf{D}$. The same fit also yields the second time derivative $\ddot{\mathbf{C}}_*(0^+)$, which ultimately gives the traffic,

$$\mathcal{T} = -\frac{1}{4} \sum_{i,j} (\mathbf{D}^{-1})^{ij} \ddot{C}_*^{ij}(0^+) = -\frac{1}{4} \text{Tr} \left[\mathbf{D}^{-1} \ddot{\mathbf{C}}_*(0^+) \right]. \quad (1)$$

1.2 Model order (Ne, Ng): richer is not safer

The fit model is a sum of N_e terms $\exp(-s_\ell t - r_\ell t^2) \times (\text{degree-}N_g \text{ polynomial in } t)$. The originally-used default, $N_e=2$, $N_g=2$ (10 free nonlinear parameters), is prone to *multimodal* behavior on systems whose correlation function has more than one relevant timescale. Checked concretely on the `linear` example (a 2×2 linear system with two distinct eigenvalues/relaxation rates): repeated fits on statistically equivalent data (same underlying process, different random seeds or trajectory lengths) gave traffic estimates $\mathcal{T}$ ranging from about 1.7 to 3.3 for a true value of 2.4. Importantly, this scatter did *not* shrink when the trajectory length was increased fivefold, ruling out ordinary statistical noise as the explanation. More likely, the nonlinear optimizer is landing in different local minima on different runs of an otherwise well-behaved fit problem. Switching to $N_e=1$, $N_g=1$ (a single $\exp(-st - rt^2)(a_0 + a_1 t)$ term, 4 free parameters) resolved this: repeated fits then agreed with each other and with the true value to within about 15%, consistently.

Recommendation: default to $N_e=1$, $N_g=1$, and move to a richer model only when $\mathcal{T}(H)$ fails to show a plateau over any reasonable H range (Sec. 1.3), as happens for `hair`; in that case, check fit stability across independent trajectory slices/seeds before trusting a single fit. `cfg.ExpPolyCfg`’s dataclass default is $N_e=1$, $N_g=1$, matching every result in the arxiv paper.

1.3 The fit window (k_1, k_2, H) is system-dependent

Only lags $t \geq t_{k_1}$ enter the fit (`WindowCfg.i1` in the code, default $k_1 = 2$), which excludes the lags most sensitive to measurement noise. The window then extends to t_{k_2} for as long as $|C_*(t)|$ stays within a fraction H of $|C_*(t_{k_1})|$ (`correl.auto_window`). There is no universal good choice of (k_1, H) : it must match how the specific system’s own correlation function actually decays – `hair`, for instance, turns negative after a short time (an oscillatory overshoot that a single exp-poly term cannot capture) and needs a correspondingly small H .

Practical procedure: sweep H over a plausible range on your own data (e.g. $H \in [0.02, 0.5]$), plot $\mathcal{T}(H)$, and look for a plateau: pick H from the flattest, most stable region rather than a fixed default. If no such plateau exists over any reasonable range, suspect a multi-timescale or oscillatory correlation function and narrow the search (smaller k_1 , smaller H range), or increase the complexity of the fitting functions by setting $N_e = 2$ and/or $N_g = 2$ or higher (Sec. 1.2). Section 6 lists the values actually used for each example.

1.4 Automating the choice: minimizing across-chunk variance is a useful default, not a guarantee

In practice we automate the sweep above and split the trajectory into independent chunks, fit $\mathcal{T}(H)$ on each chunk separately for every H in the sweep, and pick the H^* with the smallest standard deviation of $\mathcal{T}$ across chunks (`common.pick_stable_Tr` in the companion notebooks). Checked against known ground truth, this criterion lands on the actual lowest-bias H in most cases we examined.

It is not, however, a bias guarantee. Minimizing variance selects for consistency, not correctness, and the two coincide only when the estimator’s main failure mode is noise rather than a systematic fitting artifact.

Recommendation: use the across-chunk-variance minimizer as a default that removes the need to hand-tune every sweep, but when ground truth is available for even one representative parameter point, use it to confirm the selector’s choice before trusting it across an entire swept family.

2 Inflow-rate estimation

`cynar.inflow.grid_inflow` bins the trajectory onto an axis-aligned grid of cell size Δx (`side`). The n recorded states give $n - 1$ midpoints between consecutive states, binned on the grid; a cell c visited by n_c of these midpoints has occupation probability $p_c = n_c / (n - 1)$ and density $\rho_c = p_c / \mathcal{V}$ ($\mathcal{V}$ the cell volume, the product of the per-axis cell sizes), and its score $\mathbf{s}_c$ is the finite-difference gradient of $\ln \rho$ across neighboring cells – which requires flooring ρ_c at a numerical floor $\epsilon = 10^{-300}$ before taking the logarithm, since some cells receive zero visits (Section 2.1 below). Only cells with $n_c > n_{\min}$, finite score, and finite local mean velocity $\mathbf{v}_c$ (Section 4) enter the raw, unsmoothed estimators

$$\mathcal{G} = \sum_{c: n_c > n_{\min}} p_c \mathbf{s}_c \cdot (\mathbf{D} \mathbf{s}_c), \quad \sigma_{v^2} = \sum_{c: n_c > n_{\min}} p_c \mathbf{v}_c \cdot (\mathbf{D}^{-1} \mathbf{v}_c), \quad (2)$$

sharing the same binned grid, which is what makes their cell-size-stability comparison (Section 4) meaningful.

2.1 Grid-boundary score artifacts (fixed)

As just noted, a cell adjacent to an empty neighbor computes its log-density against this same floored value, $\rho \rightarrow \epsilon = 10^{-300}$. A finite-difference gradient taken across such a boundary produces an artifact of magnitude

$$g_{\text{artifact}} \approx \frac{\log(1/\epsilon)}{2 \Delta x} = \frac{690.8}{2 \Delta x}, \quad (3)$$

which is not a real score value at all, just a discretization artifact. This is normally caught by the score-clipping threshold `InflowCfg.thr_g` (default 200), which zeroes any score component whose magnitude exceeds it; *but* g_{artifact} *shrinks* as Δx grows, so past a computable crossover cell size,

$$\Delta x_c = \frac{\log(1/\epsilon)}{2 \cdot \text{thr_g}}, \quad (4)$$

($\Delta x_c \approx 1.73$ for the default `thr_g=200`) the artifact slips *under* the clip and survives into the final sum, where it gets squared and weighted by $\mathbf{D}$. Just past this crossover, this can produce an artificial explosion in the $\mathcal{G}$ estimate (a two-order-of-magnitude blow-up was observed for `hair`), unless `thr_g` is lowered accordingly.

Fix, now in the package: `cynar.grid.sparse_neighbor_mask` identifies grid cells whose finite-difference score touches a sparse or empty neighbor (using the same occupancy threshold `n_min` used elsewhere) and zeroes their score *outright*, before any magnitude-based clipping; this is independent of cell size or D ’s magnitude by construction, unlike a fixed clip. `thr_g` is retained purely as a secondary safety net for other sources of noise (e.g. a cell count just above `n_min` still giving a noisy density estimate); do not rely on it alone to catch boundary artifacts.

2.2 The safety-net threshold does not scale with D

Even where the boundary-artifact fix above does not apply, remember that `thr_g` bounds each score *component*, while the inflow-rate integral sums $D_{ii} \times (\text{score}_i)^2$ over all dimensions. For a system with large D entries and/or high dimension, a small number of cells with scores near (but under) the clip can still dominate the sum once weighted by D and summed over many dimensions. If you see occasional wildly-off inflow-rate estimates that don’t fit the boundary-artifact mechanism above, check whether this is the cause: a D -aware threshold (e.g. bounding each cell’s contribution $D_{ii} g_i^2$ directly) would be a more robust general fix, not yet implemented.

2.3 Choosing Δx for a swept parameter family

A single fixed cell size Δx does not automatically suit every point of a swept parameter family.

Fix: rescale Δx per parameter point by the ratio of that point’s spread to the reference point’s, using a robust per-dimension estimate of spread, e.g.

`side0( $\beta$ ) = sideeq × gmean(std( $X_\beta$ ))/gmean(std( $X_{\text{ref}}$ ))` (geometric mean of the per-axis standard deviation, to get a single scalar scale factor from a possibly anisotropic spread). This resolved the blow-up and improved most points across the sweep, without needing to change the tested range of cell-size multipliers itself.

Recommendation: for any swept parameter family, do not assume one cell size fits the whole family: check how much the stationary distribution’s spread varies across the sweep, and rescale Δx accordingly if it varies by more than $\sim 50\%$.

2.4 No universal blind heuristic for Δx without ground truth

Since a Δx selector needs a variance-like signal that is not always trustworthy, we tested a purely “blind” heuristic that needs no sweep at all: set Δx to roughly one-tenth of the trajectory’s excursion in each direction (e.g. `std( $X$ )/10` per axis, rounded to a convenient number). Checked against three example systems, this heuristic worked well for `hair`, was roughly right (same order of magnitude as the actual best Δx) for `Lorenz`, but was off by a factor of 4–5 (recommending a Δx far too fine) for `linear`. **Conclusion: no universal blind cell-size rule was found; some form of sweep against a stability or ground-truth signal remains necessary.**

2.5 A starting point for the sweep, on unfamiliar data

Given the conclusion above, `cynar.inflow.propose_cell_sizes` does not try to guess Δx outright; instead it proposes a base cell size Δx_0 , set from each dimension’s interquartile range divided by a target number of bins per axis (default 10), together with a handful of multipliers ($0.5\text{--}2\times$ by default) to sweep around it, in the exact form expected by `sweep_cellsize`. This, along with `cynar.data_io.load_trajectory` (for reading a trajectory file, optionally with a leading time column, and inferring dt) and an automatic skip of this whole grid/inflow step above a configurable dimension, is wired into `notebooks/template_your_data.ipynb`: a generic template for running

`cynar` on your own data rather than one of the shipped examples. As with any starting point, still inspect the resulting stability sweep rather than trusting Δx_0 on its own.

3 Smoothing

The optional Nadaraya-Watson smoothing step produces $\tilde{\mathcal{G}}$ ² (and the v^2 -method’s $\sigma_{v^2, \text{smooth}}$, Section 4) by locally averaging the raw binned fields over neighboring cells. Two independent bugs were found while building it; both mattered, and fixing only one was not enough. Section 3.1 gives the construction as it stands now, fixes included; Sections 3.2–3.3 are the bug-diagnosis narrative behind it.

3.1 The smoothed-field construction

The smoothing is a Nadaraya-Watson, Gaussian-kernel regression, restricted to occupied cells ($n_c > n_{\min}$) only, so that a thin or curved density support – a chaotic attractor’s, in particular – is not diluted by unvisited neighboring cells (Section 3.3). All fields are smoothed with the same bandwidth $h = Q \Delta x$, a multiple of the grid’s own cell size (Section 3.2), so that the smoothing radius expressed in cells stays comparable across systems and cell sizes; every example in the arxiv paper uses $Q = 0.5$. Cell c' enters the neighborhood of cell c with Gaussian weight

$$w_{cc'} = \exp \left[-\frac{1}{2} \sum_i \left((x_c^i - x_{c'}^i) / h^i \right)^2 \right], \quad (5)$$

kept only while $w_{cc'} \geq w_{\min}$ (equivalently, within a radius of $\sqrt{2 \ln(1/w_{\min})}$ bandwidths); we use $w_{\min} = e^{-3}$ throughout (`SmoothingCfg.w_min`). For any field g_c defined on occupied cells, denote by $\mathcal{S}[g]_c = \sum_{c'} w_{cc'} g_{c'} / \sum_{c'} w_{cc'}$ its smoothed version.

The smoothed density is $\bar{\rho} = \mathcal{S}[\rho]$. We do not smooth $\mathbf{s}$ and $\mathbf{v}$ directly; instead each is first combined with the square root of the smoothed density, since $\int d\mathbf{x} \rho \mathbf{s} \cdot \mathbf{D} \mathbf{s} = \int d\mathbf{x} (\sqrt{\rho} \mathbf{s}) \cdot \mathbf{D}(\sqrt{\rho} \mathbf{s})$ (and similarly for $\mathbf{v}$), which absorbs the density weight into the field so that the final estimators below become plain sums over cells:

$$\tilde{\mathbf{s}} = \mathcal{S}[\sqrt{\bar{\rho}} \mathcal{S}[\mathbf{s}]], \quad \tilde{\mathbf{v}} = \mathcal{S}[\sqrt{\bar{\rho}} \mathbf{v}]. \quad (6)$$

That is, the density-weighted field is smoothed once in both cases, and the raw score is also smoothed once before being weighted³. The smoothed estimators are then

$$\tilde{\mathcal{G}} = \mathcal{V} \sum_{c: \bar{\rho}_c > \rho_{\min}} \tilde{\mathbf{s}}_c \cdot (\mathbf{D} \tilde{\mathbf{s}}_c), \quad \sigma_{v^2, \text{smooth}} = \mathcal{V} \sum_{c: \bar{\rho}_c > \rho_{\min}} \tilde{\mathbf{v}}_c \cdot (\mathbf{D}^{-1} \tilde{\mathbf{v}}_c), \quad (7)$$

with $\rho_{\min} = n_{\min} / [(n-1)\mathcal{V}]$ the same minimal-occupancy floor as the raw grid (Section 2).

3.2 Bandwidth must be tied to cell size, not to the raw data’s spread (fixed)

The smoothing bandwidth used to be $Q \times \text{IQR}(\text{data})$, a fraction of the trajectory’s own interquartile spread, with *no relationship at all* to the chosen grid cell size Δx . Since Δx is chosen per system

²Here, the tilde symbol in $\tilde{\mathcal{G}}$ was used originally in the code to denote smoothed quantities and does not refer to quantities obtained from a subsets of coordinates, as in the arxiv paper.

³Since two Gaussian smoothings with the same kernel are approximately equivalent to one with a bandwidth larger by $\sqrt{2}$, the score ends up effectively smoothed on a somewhat larger scale than the velocity, though both use the same Q .

and per sweep point independently of the data’s raw physical scale, the resulting smoothing radius *in cells* (radius $\approx 2.45 \times Q \times \text{IQR} / \Delta x$, where 2.45 is the neighbor-inclusion radius set by $w_{\min} = e^{-3}$) varied wildly across examples for the same nominal Q : about 1.6–2.2 cells for `linear`, but 17–24 cells for `hair` (whose IQR happens to be numerically large in its physical units). At that width, smoothing averages the score field, which is roughly antisymmetric around the density mode, down to nearly zero: this is not a smoothed estimate of $\mathcal{G}$, it is an over-smoothed one (checked: `hair`’s $\tilde{\mathcal{G}}$ was ≈ 36 –70 against a true value of ≈ 145 before the fix below).

Fix, now in the package: bandwidth $h = Q \Delta x$, as given in Section 3.1, in both `inflow.grid_inflow` and `v2method.grid_v2`. Default $Q = 0.5$ for all of `rho`, `nu`, `g` (`cfg.InflowCfg.Q`), tuned empirically across `linear`, `hair`, `Lorenz` (Section 6 gives the full picture, including where $Q = 0.5$ is not quite enough).

3.3 Smoothing over sparse or fractal supports needs occupied-cells-only neighbors (fixed, with a residual limitation)

Even after the fix above, `Lorenz`’s smoothed $\tilde{\mathcal{G}}$ remained badly suppressed, and got *worse*, not better, as the smoothing bandwidth increased. Diagnosis: the neighbor search for smoothing ran over the entire rectangular grid (the Cartesian product of per-axis bin edges), not just the occupied cells. For a distribution concentrated on a thin or curved region of state space, a chaotic attractor’s fractal support in particular, most of that rectangular bounding box is empty. Checked concretely: `Lorenz`’s grid was only $\approx 10\%$ occupied versus $\approx 48\%$ for `hair` at comparable resolution, and `Lorenz`’s neighbor counts for the smoothing kernel came out about $3\times$ larger than `hair`’s for the *same* nominal bandwidth, almost entirely extra empty (“phantom”) cells, not real structure. Averaging those in as literal zero-valued data dilutes the smoothed field.

Fix, now in the package: the neighbor search, and the smoothed quantities themselves, are restricted to occupied cells only (`counts > n_min`, already reflected in Section 3.1); results are scattered back onto the full grid only afterward, for downstream plotting.

Residual limitation (not yet fully resolved): even with both fixes, `Lorenz`’s smoothed $\tilde{\mathcal{G}}$ remains the least accurate of the systems tested, roughly 30–70% below the true value across a cell-size sweep, versus 2–15% for `linear` and `hair`. This looks like a dimensionality effect that has not yet been fully characterized: a fixed bandwidth multiplier Q admits a ball of $\sim (2.45 Q)^N$ cells in N dimensions, so the *effective* neighbor count for a given Q grows sharply with dimension even after restricting to occupied cells: a 3-dimensional system like `Lorenz` likely needs a smaller Q than a 2-dimensional one for a comparable effective smoothing radius, and we have not yet derived or tuned a per-dimension rule. This affects only the *smoothed* inflow-rate/ v^2 -method variants; the raw (unsmoothed) grid estimate, and the grid-free traffic-only route used for high-dimensional systems (arxiv paper, `Lorenz-96` example), are unaffected.

4 The v^2 -method

`cynar.v2method` implements the comparison baseline used throughout the arxiv paper: $\sigma_{v^2} = \langle \mathbf{v} \cdot (\mathbf{D}^{-1} \mathbf{v}) \rangle$, where $\mathbf{v}(\mathbf{x}) = \mathbf{A}(\mathbf{x}) - \mathbf{D} \mathbf{s}(\mathbf{x})$ is the local mean velocity (equivalently, the steady-state probability current divided by the density), following Gnesotto, Mura, Gladrow & Broedersz’s review of broken-detailed-balance estimators. It shares grid/binning code (`cynar.grid`) with `inflow.grid_inflow`, so both estimators read off *exactly the same cells* for a given system and Δx : this is what makes a cell-size-stability comparison between the two methods meaningful rather than an artifact of using different discretizations. Both smoothing fixes in Section 3 apply identically to

`v2method.grid_v2`'s smoothed variant, $\sigma_{v^2, \text{smooth}}$; the boundary-artifact fix (Section 2.1) is inflow-specific (it concerns the score, which v^2 does not use) and has no v^2 -method analog.

5 Partial observation

All estimators can be applied to any subset Ω of the recorded coordinates. The block D_Ω follows from the correlations of $\mathbf{x}_\Omega$ and is always invertible, and $\max(4\mathcal{T}^\Omega + \mathcal{G}^\Omega, 0)$ bounds σ from below for any D . When D_Ω is block diagonal, the bound can also be applied to each block separately, and the larger of the two results can be used. The v^2 method applied to $\mathbf{x}_\Omega$ yields $\sigma_{v^2}^\Omega$, which is a lower bound as well but vanishes when a single coordinate is observed, and whose finite-sample bias can push it above the bound at small cell sizes. For high-dimensional data, where the grid becomes infeasible, the traffic-only bound $\max(4\mathcal{T}^\Omega, 0)$ remains available.

In code, this only means running the usual pipeline on the observed columns `X[:, subset]` alone: `traffic.compute_traffic` for D_Ω and $\mathcal{T}^\Omega$, then `inflow.grid_inflow` or `v2method.grid_v2` for $\mathcal{G}^\Omega$ or $\sigma_{v^2}^\Omega$, always refitting D_Ω from these coordinates' *own* correlations rather than slicing it out of a full-system fit. `common.honest_partial_estimate` wraps exactly this, and `03_lorenz.ipynb`'s partial-observation cells work through it end to end.

6 Per-system configuration reference

Configuration actually used to produce every figure in the arxiv paper. All traffic fits used `Ne=1`, `Ng=1` (Section 1.2) except where noted.

System	traffic (k_1, H)	inflow <code>side₀</code>	smoothing Q	<code>thr_g</code>
linear	2, 0.05–0.5	(0.5, 0.3)	0.5	200
hair	2, 0.02–0.1	(0.5, 0.5)	0.5	200
Lorenz ($N=3$)	2, 0.05–0.4	(0.5, 0.5, 0.5)	0.5	200
Lorenz96 ($N=4$ –25)	2, 0.05	<i>n/a (traffic only)</i>	<i>n/a</i>	<i>n/a</i>

7 Open items

- **Smoothing's residual dimension-dependence** (Section 3.3): no per-dimension rule for Q derived yet, only the qualitative $\sim Q^N$ scaling argument.
- **A D-aware `thr_g`** bounding each cell's contribution to the inflow-rate sum directly (rather than each score component's raw magnitude) would be a more robust general fix than the current fixed threshold; not implemented, since the boundary-artifact fix (Section 2.1) already addresses the specific failure mode found in practice.